Learning Role-Aware Submodular Cut Selection for Dual-Source Rotary Cluster-Tool Scheduling

Anonymous submission

Abstract

We study a rotary semiconductor cluster tool in which product wafers and reusable process-enabling carrier (PEC) wafers enter from different stores and share robots, load locks, and process chambers. Its four-wafer rotary recipe and pair-exchange recipe couple discrete assignment with continuous-time resource coordination. We formulate an end-to-end mixed-integer model that includes serial alignment, loaded and empty robot motion, load-lock capacity, PEC reuse, chamber rotation, and cleaning. At the solver level, standard cut-quality features cannot distinguish candidates with similar efficacy but different mixtures of discrete and continuous support. Our key observation is that absolute row coefficients, grouped by solver-observable variable roles, provide this missing information without application labels or variable-name parsing. We append ten such features to the thirteen HEM features, predict cut cardinality and order hierarchically, and complete a policy-anchored subset by greedily maximizing a monotone role-coverage objective. The objective combines cut quality, concave role coverage, and facility-location representativeness, giving the structural layer an explicit approximation guarantee rather than a fixed quota heuristic. Because the method selects only SCIP-generated cuts, it does not change the feasible set. Small exact solves validate the formulation and expose a computational cliff on the mixed $8 + 8$ case; multi-seed selector results are not yet available.

Introduction

A schedule for a semiconductor cluster tool must specify more than a chamber order. It must decide when each wafer leaves storage, which robot carries it, which load-lock slot it occupies, and when a chamber exposes the required side. We consider a tool with two material sources: product wafers start at load ports, whereas process-enabling carrier (PEC) wafers start at a separate store and can be reused. Both flows share an atmospheric robot (ATR), a vacuum robot (VTR), two load locks, and two rotary chambers. The objective is the return time of the last product wafer.

The equipment combines two recipes whose timing structures are different. The 4×1 recipe fills the two sides of a four-pocket chamber before processing. The 2×2 recipe forms a contiguous exchange chain in which an incoming pair replaces an outgoing pair between consecutive exposures. Existing cluster-tool studies address cyclic operation, concurrent wafer types, or non-cyclic robot sequencing (Ahn and Kim 2025; Kim and Lee 2025; Li and Yang 2025; Yang et al. 2025). Our setting adds a finite reusable PEC stream, two rotary recipes, a single-slot aligner, and cleaning batches. Aggregating any of these operations can create a physically impossible schedule: a pair cannot occupy the aligner together, a robot cannot perform two loaded moves without repositioning, and the same PEC wafer cannot support overlapping chamber jobs.

The detailed formulation is difficult to solve because it couples assignment binaries with a large continuous-time disjunctive schedule. SCIP strengthens such models by generating cutting planes and retaining a subset at each separation round (Bestuzheva et al. 2023). Learned selectors estimate a score for each cut (Huang et al. 2022), predict both cardinality and order (Wang et al. 2023), or adapt the weights of SCIP's hybrid score (Turner et al. 2023). These methods describe efficacy, violation, support, and objective alignment well. They do not, however, explicitly describe whether a cut is concentrated on binary decisions, continuous auxiliaries, or a coupling between the two. Consequently, a high-quality prefix can contain several cuts with similar variable support while omitting a different role that is also active in the relaxation.

We address this issue with role-aware hierarchical cut selection. For every candidate row, we distribute the absolute coefficient mass over variable roles available directly from SCIP; no variable-name parser or application label is used. These role features augment a HEM-style policy that first chooses how many cuts to retain and then orders the candidates. Because an ordered neural decoder can still repeat the same structural pattern, we apply a policy-anchored submodular completion layer that balances quality, role coverage, and candidate representativeness. Figure 1 shows the complete path from the scheduling MIP to the selected SCIP cuts.

This paper makes three technical contributions. First, it gives an end-to-end mixed-integer model of dual-source rotary scheduling, including operations that are commonly hidden inside aggregate transfer or chamber durations. Second, it introduces a name-invariant cut representation that records

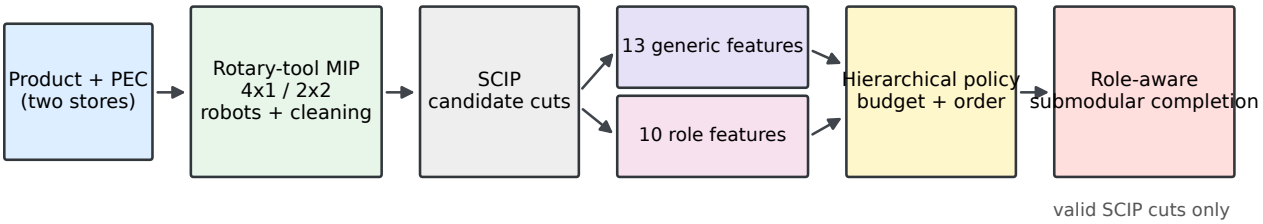

Figure 1: Overview. The scheduling model supplies a difficult mixed discrete–continuous instance, but the cut representation itself is domain independent. SCIP produces valid candidate cuts; the learned policy chooses a budget and an order from generic and variable-role features; role-submodular completion balances quality, coverage, and representativeness before returning the selected prefix.

how a cut acts on discrete and continuous variable roles and can therefore be computed for arbitrary SCIP models. Third, it combines learned cardinality and ordering with a monotone role-aware submodular objective whose greedy completion has a standard approximation guarantee, while retaining SCIP's cut-validity guarantees. The empirical study is designed to separate these contributions through a 13D HEM baseline, a 23D feature-only variant, and the full submodular policy.

Related Work

Cluster-tool scheduling. Cluster-tool schedules have been constructed with cyclic robot rules, Petri nets, mathematical programming, and reinforcement learning. Recent MIP formulations improve non-cyclic scheduling by representing precedence relations directly (Ahn and Kim 2025), while learning-based work chooses robot and wafer-release actions without a fixed cyclic sequence (Kim and Lee 2025). Other studies handle concurrent wafer types and residency-time constraints (Li and Yang 2025; Yang et al. 2025). These models motivate explicit robot and module timing, but they do not cover the combination studied here: product and reusable PEC sources, two rotary recipes, and cleaning within one finite-horizon schedule.

Learning inside exact optimization. Machine learning has been used to replace or guide expensive decisions in exact solvers. Examples include branching policies trained across MIP families (Manchanda and Ranu 2023), reinforcement learning from reconstructed branch-and-bound trajectories (Parsonson, Laterre, and Barrett 2023), and learned surrogates inside cutting-plane algorithms (Mana et al. 2024). Scheduling work has likewise combined learned predictions with optimization layers that recover feasibility (Kotary, Fioretto, and Van Hentenryck 2022). Our learned component does not construct a schedule or a cut. It acts only at SCIP's cut-selection interface, leaving the model and the validity of generated inequalities unchanged.

Learning to select cuts. Cut selection must balance relaxation improvement against LP and callback overhead. Huang et al. learn scores for individual cuts (Huang et al. 2022); HEM treats the retained cardinality and ordered subset as coupled decisions (Wang et al. 2023); and Adaptive Cut Selection predicts the weights of SCIP's hybrid score (Turner et al. 2023). The broader design space is reviewed by Deza and Khalil (Deza and Khalil 2023). We retain HEM's separation between cardinality and ordering, but add solver-observable variable roles and a policy-anchored submodular completion layer. In our comparisons, HEM receives exactly the original 13 generic features; ACS-Tuned is distinguished from full ACS, which requires held-out instance-level weight predictions.

Dual-Source Rotary Scheduling Model

Material flow. Let $W = W^{4\times1} \cup W^{2\times2}$ be the product wafers, Q the reusable PEC tokens, and $C = \{\mathrm{CH2}, \mathrm{CH3}\}$ the rotary chambers. A product wafer follows

$$\mathrm{LP} \to \mathrm{ATR} \to \mathrm{AL} \to \mathrm{LL}^{up} \to \mathrm{VTR}$$
$$\to \mathrm{CH} \to \mathrm{LL}^{low} \to \mathrm{ATR} \to \mathrm{LP},$$

whereas a PEC wafer follows $\mathrm{PECStore} \to \mathrm{VTR} \to \mathrm{CH} \to \mathrm{VTR} \to \mathrm{PECStore}$. Product wafers of the same recipe are paired. If a recipe contains an odd final wafer, that wafer is paired with one PEC token rather than with a fictitious empty wafer.

Rotary recipes. An active 4×1 batch assigns one pair to each chamber side. The chamber executes front load, half rotation, back load, processing, back unload, half rotation, and front unload. If only one product pair remains, the unused side is filled by a PEC pair; such a filler is allowed only in the tail batch of a chamber sequence. For 2×2, product pairs form a contiguous prefix chain between a PEC head and a PEC tail. Each product pair appears in two adjacent processing exposures, and the interval between them contains the physical pair exchange. This construction permits the necessary bridge wait but does not create an "empty wafer" decision.

Assignments and time intervals. Let P be the product-pair set, B_c the candidate 4×1 batches on chamber c, and

K_c the candidate 2×2 positions. Binary variables x^F and x^M assign each pair exactly once:

$$\sum_{c \in C} \left(\sum_{b \in B_c} \sum_{h=1}^{2} x^F_{pbhc} + \sum_{k \in K_c} x^M_{pkc} \right) = 1, \quad p \in P. \quad (1)$$

Each physical action a has start s_a, end e_a, and duration d_a. For two optional tasks a and b using the same unary resource, an order variable y_{ab} enforces

$$s_b \geq e_a + \delta_{ab} - M(1 - y_{ab}), \quad (2)$$
$$s_a \geq e_b + \delta_{ba} - M y_{ab}, \quad (3)$$

where δ is the endpoint-dependent empty reposition time. The same construction coordinates ATR, VTR, chamber windows, and load-lock slots.

Physical details that affect feasibility. AL has one slot, so the two wafers carried by ATR are calibrated serially: ATR places one wafer, removes it after calibration, and only then places its companion. ATR and VTR tasks include pickup, loaded travel, placement, and the empty move required before the next pickup. Upper and lower load locks have two explicit slots. A PEC job occupies one chamber-bound token from VTR load start through VTR unload end, which prevents overlapping reuse. After a configurable number of full-chamber processing cycles, cleaning is represented as a pure-PEC rotary batch with the same VTR and chamber-side semantics as production.

Objective. The primary objective minimizes the return makespan

$$\min C_{\max}, \qquad C_{\max} \geq C^{\text{return}}_w \quad (w \in W). \quad (4)$$

For schedule visualization, an optional second phase fixes $C_{\max}$ and minimizes convex penalties on avoidable waiting in chambers, load locks, and wafer routes. This phase is lexicographic: it cannot trade a worse makespan for a visually smoother schedule. It is disabled in the generic cut-selection benchmarks.

Role-Aware Hierarchical Cut Selection

At separation round t, SCIP supplies candidate rows $\mathcal{C}_t$ and a maximum retained cardinality. Our selector executes four steps: compute cut features, predict a budget, predict an ordered subset, and complete it under a role-aware submodular objective. Only the final ordering and retained prefix are returned; cut coefficients and right-hand sides are never modified.

Generic cut features. The 13D HEM representation contains objective parallelism, efficacy, support, integral support, normalized violation, four statistics of the cut coefficients, and four statistics of objective coefficients. These measurements describe immediate cut quality and scaling. They do not identify which variable types contribute that support.

Table 1: Ten role-aware features appended to the 13 generic features. Each mass is normalized by $\sum_j |a_{ij}| + \epsilon$.

Count	Features
5	Coefficient mass on B, I, C_{obj}, C_{aux}, and O
1	Mass on variables with finite global lower and upper bounds
1	Positive-coefficient mass
1	Disc.–cont. coupling $4(p_{iB} + p_{iI})(p_{iC_{\text{obj}}} + p_{iC_{\text{aux}}})$
1	Largest non-residual role mass
1	Role entropy $-\sum_g p_{ig} \log p_{ig} / \log \lvert\mathcal{G}\rvert$

Variable-role coefficient mass. For a candidate cut i with row $\sum_j a_{ij} x_j \leq b_i$, let

$$\mathcal{G} = \{B, I, C_{\text{obj}}, C_{\text{aux}}, O\}$$

denote binary, general-integer, objective-active continuous, continuous auxiliary, and unknown roles. The role mass is

$$p_{ig} = \frac{\sum_{j:\, x_j \in g} |a_{ij}|}{\sum_j |a_{ij}| + \epsilon}. \quad (5)$$

All role assignments come from SCIP variable metadata. In particular, no scheduling-specific variable name is inspected.

Table 1 lists the remaining row-geometry features. The coupling term is zero when a cut acts only on one side of the discrete–continuous split and is largest for balanced support. The dominant mass and entropy distinguish a focused cut from one that mixes many roles. The complete candidate vector has 23 dimensions and is invariant to variable renaming.

Budget and ordered subset. A high-level policy maps the candidate feature set to a retained fraction ρ_t, which is clipped by SCIP's limit and an implementation cap to obtain k_t. A pointer network then samples or greedily decodes an ordered sequence of candidates. Training minimizes the observed primal-dual integral through REINFORCE updates with an exponential moving-average baseline. Independent SCIP processes collect complete rollouts, and their trajectories are aggregated before each parameter update. We therefore refer to the implementation as parallel hierarchical policy-gradient training: environment rollouts are parallel, whereas parameter updates are centralized.

Policy-anchored role-submodular completion. The neural order may devote most of its budget to one structural pattern. We retain its first $\lceil \gamma k_t \rceil$ candidates as anchors A_t and form an expansion pool R_t of at most αk_t candidates by appending rows with high normalized structural quality q_i. For $S \subseteq R_t$, define

$$F_t(S) = \sum_{i \in S} (.35 q_i + .20 r_i) + .25 \sum_{g \in \mathcal{G}} d_g \sqrt{\sum_{i \in S} p_{ig}} + \frac{.20}{|R_t|} \sum_{u \in R_t} \max_{i \in S} s_{ui}, \quad (6)$$

where r_i is the normalized neural rank prior, d_g is role demand in R_t, and $s_{ui} \in [0, 1]$ is the clipped cosine similarity between generic–role cut descriptors. The first term preserves immediate quality and the learned order. The concave second term gives diminishing returns to repeated role mass, while the facility-location term rewards a subset that represents the candidate pool without an explicit pairwise penalty.

Starting from A_t, we repeatedly add the candidate with largest marginal gain $F_t(S \cup \{i\}) - F_t(S)$ until $|S| = k_t$. All coefficients in Eq. (6) are nonnegative; hence its modular, concave-over-modular, and facility-location components are monotone submodular. For fixed anchors, the residual objective $F_t(A_t \cup T) - F_t(A_t)$ remains normalized, monotone, and submodular over $R_t \setminus A_t$. Cardinality-constrained greedy therefore obtains at least a $(1 - 1/e)$ fraction of the best residual gain achievable inside R_t (Nemhauser, Wolsey, and Fisher 1978). We use $\alpha = 2$ and $\gamma = .25$, selected on validation and fixed for every family. The layer only chooses among SCIP-generated valid cuts and therefore cannot remove any integer-feasible solution.

Experimental Protocol

The experiments test three claims separately. First, role features should improve cut selection over a 13D policy under the same training and solver budgets. Second, submodular completion should add value beyond the features alone. Third, a name-invariant representation should remain usable outside the scheduling formulation, although transfer gains are not assumed.

Data. The scheduling generator uses disjoint size and recipe-ratio splits. Training contains pure and mixed instances with at most 12 product wafers; validation contains 16-wafer instances; test contains 24–32 wafers and recipe ratios not seen in training. For cross-family tests, we generate set cover, multidimensional knapsack, capacitated facility location, and maximum-weight independent set instances at three scales. MIPLIB 2017 provides a heterogeneous external set (Gleixner et al. 2021).

Baselines and ablations. We compare SCIP Default, ACS-Tuned, full ACS when held-out instance predictions are available, HEM with a separately trained 13D checkpoint, and the proposed 23D policy. HEM and our policy use identical instances, optimizer budgets, candidate caps, and random seeds. The core ablation adds the 10 role features while disabling submodular completion; the full model then enables Eq. (6). Additional ablations vary candidate cap, retained cardinality, greedy versus beam decoding, expansion factor α, anchor ratio γ, and each submodular component.

Metrics and statistics. The primary anytime metric is primal-dual integral (PDI). We also record shifted geometric mean runtime, final gap, node count, incumbent rate, optimality rate, feature-extraction time, and neural inference time. Scheduling runs additionally report makespan and robot/chamber utilization. All comparisons are paired by instance and SCIP seed. The formal study uses five training seeds and at least ten SCIP seeds, with bootstrap 95% confidence intervals, paired Wilcoxon tests, and Holm correction across families.

Table 2: Recorded formulation-validation runs. The mixed $8 + 8$ instance has an incumbent at the 60-second limit but retains a large gap.

Instance	Vars	Cons.	$C_{\max}$	Status
Pure 4×1 (4)	684	1,645	388	optimal
Pure 2×2 (4)	876	1,959	324	optimal
Mixed (4+4)	1,182	2,765	508	optimal
Mixed (8+8)	2,932	7,995	1,150	limit

Implementation. Experiments use Python 3.13.5, PySCIPOpt 6.1.0, NumPy 2.1.3, and PyTorch 2.11.0. Candidate pools are capped at 128 and retained prefixes at 16 unless chosen otherwise on validation. The pointer network has a 128-dimensional hidden state and one glimpse. The 13D and 23D checkpoints are trained separately; a schema tag prevents legacy 23D weights from being loaded after the feature semantics changed.

Current Validation

Formulation checks. Table 2 summarizes deterministic validation runs with two chambers, 110-second 4×1 processing, 50-second 2×2 exposures, and reusable PEC tokens. These runs test model connectivity and the expected computational regime; they do not compare learned selectors.

The two pure four-wafer instances solve at the root in under one second. The mixed $4+4$ instance requires 797 nodes and about 11 seconds. The mixed $8 + 8$ instance explores 7,779 nodes in 60 seconds and ends with a 99.98% relative gap. The change is not evidence that a learned selector will succeed, but it identifies the mixed regime in which cut management can be measured.

Status of learned-policy results. The HEM training is still running, and the 23D policy must be retrained because the postprocessor used during its rollout transition changed from fixed quota repair to Eq. (6). Accordingly, this draft makes no claim of improvement over SCIP, ACS, or HEM. The final submission must replace this paragraph with frozen paired results, the feature-only and submodular ablations, cross-scale tests, cross-family tests, and callback-overhead measurements.

Limitations

The five variable roles are intentionally coarse. They do not encode constraint-graph topology, indicator semantics, symmetry, or cut provenance. The submodular component weights, expansion factor, and anchor ratio are fixed globally and require validation-only sensitivity analysis before they can be treated as robust defaults. The scheduling instances use engineering parameters rather than production traces, and the formulation-validation table predates the final multi-seed cut study. Finally, policy updates are synchronized after rollout collection; asynchronous optimization is outside the present implementation.

Conclusion

This work connects a detailed dual-source rotary scheduling model with a solver-level learning problem. The scheduling formulation makes alignment, robot repositioning, load-lock capacity, PEC reuse, rotary exchange, and cleaning explicit. The cut selector then describes each SCIP candidate by generic quality and name-invariant variable-role mass, predicts a cardinality and order, and performs policy-anchored submodular completion without changing any inequality. The present evidence validates the formulation and the implementation interfaces. Performance conclusions depend on the frozen multi-seed comparison now in progress.

References

Ahn, J.; and Kim, H. J. 2025. A Novel Mixed Integer Programming Model with Precedence Relation-Based Decision Variables for Non-Cyclic Scheduling of Cluster Tools. *IEEE Transactions on Automation Science and Engineering*, 22: 2893–2908.

Bestuzheva, K.; Besançon, M.; Chen, W.-K.; Chmiela, A.; Donkiewicz, T.; van Doornmalen, J.; Eifler, L.; Gaul, O.; Gamrath, G.; Gleixner, A.; Gottwald, L.; Graczyk, C.; Halbig, K.; Hoen, A.; Hojny, C.; van der Hulst, R.; Koch, T.; Lübbecke, M. E.; Maher, S. J.; Matter, F.; Mühmer, E.; Müller, B.; Pfetsch, M. E.; Rehfeldt, D.; Schlein, S.; Schlösser, F.; Serrano, F.; Shinano, Y.; Sofranac, B.; Turner, M.; Vigerske, S.; Wegscheider, F.; Weninger, D.; and Witzig, J. 2023. Enabling Research through the SCIP Optimization Suite 8.0. *ACM Transactions on Mathematical Software*, 49(2): 22:1–22:21.

Deza, A.; and Khalil, E. B. 2023. Machine Learning for Cutting Planes in Integer Programming: A Survey. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence*, 6592–6600.

Gleixner, A.; Hendel, G.; Gamrath, G.; Achterberg, T.; Bastubbe, T.; Berthold, T.; Christophel, P. M.; Jarck, K.; Koch, T.; Linderoth, J.; Lübbecke, M.; Mittelmann, H. D.; Ozyurt, D.; Ralphs, T. K.; Salvagnin, D.; and Shinano, Y. 2021. MIPLIB 2017: Data-Driven Compilation of the 6th Mixed-Integer Programming Library. *Mathematical Programming Computation*, 13: 443–490.

Huang, Z.; Wang, K.; Liu, F.; Zhen, H.-L.; Zhang, W.; Yuan, M.; Hao, J.; Yu, Y.; and Wang, J. 2022. Learning to Select Cuts for Efficient Mixed-Integer Programming. *Pattern Recognition*, 123: 108353.

Kim, H. J.; and Lee, J. H. 2025. Scheduling Cluster Tools for Concurrent Processing: Deep Reinforcement Learning with Adaptive Search. *IEEE Transactions on Automation Science and Engineering*, 22: 3783–3796.

Kotary, J.; Fioretto, F.; and Van Hentenryck, P. 2022. Fast Approximations for Job Shop Scheduling: A Lagrangian Dual Deep Learning Method. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, 7239–7246.

Li, X.; and Yang, W. 2025. Cyclic Scheduling of Multi-Type Wafers Concurrent Processing in Single-Arm Cluster Tools with Residency Time Constraints. *Expert Systems with Applications*, 269: 126443.

Mana, K.; Acero, F.; Mak, S.; Zehtabi, P.; Cashmore, M.; Magazzeni, D.; and Veloso, M. 2024. Accelerating Cutting-Plane Algorithms via Reinforcement Learning Surrogates. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, 20786–20793.

Manchanda, S.; and Ranu, S. 2023. LIMIP: Lifelong Learning to Solve Mixed Integer Programs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, 9047–9054.

Nemhauser, G. L.; Wolsey, L. A.; and Fisher, M. L. 1978. An Analysis of Approximations for Maximizing Submodular Set Functions—I. *Mathematical Programming*, 14: 265–294.

Parsonson, C. W. F.; Laterre, A.; and Barrett, T. D. 2023. Reinforcement Learning for Branch-and-Bound Optimisation Using Retrospective Trajectories. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, 4061–4069.

Turner, M.; Koch, T.; Serrano, F.; and Winkler, M. 2023. Adaptive Cut Selection in Mixed-Integer Linear Programming. *Open Journal of Mathematical Optimization*, 4: 1–28.

Wang, Z.; Li, J.; Wang, Z.; Kuang, Y.; Yuan, M.; Zeng, J.; Zhang, Y.; and Wu, F. 2023. Learning Cut Selection for Mixed-Integer Linear Programming via Hierarchical Sequence/Set Model. In *International Conference on Learning Representations*.

Yang, F.; Li, C.; Zhen, L.; Zhang, C.; and Wu, N. 2025. Scheduling Residency Time-Constrained Single-Armed Cluster Tools with Two-Wafer Types via Mixed Integer Programming Model. *Expert Systems with Applications*, 288: 128209.